

Lab 4: VibeCoding — LLM Chat History

Name: Dhanush S SRN: PES1UG24AM360 Section: F

Repo: 60_number_guess | LLM used: Claude (claude.ai)

<https://claude.ai/share/b2955fc2-4dfd-4f2a-b141-df3b62a2eb82>

Prompt 1 — You

attached: game_engine.py 90 lines PY

This is game_engine.py from a Pygame number guessing game. Submitting an empty input box crashes with a ValueError at int(self.input_box.text).

Fix it so an empty submission shows a warning in the feedback area instead of crashing, and does NOT increment the attempt counter. Return only the changed submit_guess method.

Claude

```
def submit_guess(self):
    if self.game_won:
        return

    text = self.input_box.text.strip()
    if not text:
        self.feedback_msg = "Please enter a number first!"
        self.feedback_color = (255, 200, 60)
        return

    guess = int(text)

    self.attempts += 1
    self.input_box.clear()

    if guess < self.secret_number:
        self.feedback_msg = f"TOO LOW! (Guess was {guess})"
        self.feedback_color = (80, 160, 240)
    elif guess > self.secret_number:
        self.feedback_msg = f"TOO HIGH! (Guess was {guess})"
        self.feedback_color = (240, 100, 80)
    else:
        self.feedback_msg = f"CORRECT! Found in {self.attempts} attempts."
        self.feedback_color = (80, 220, 90)
        self.game_won = True
```

An empty or whitespace-only submission now returns early with an amber warning, before self.attempts is incremented. This assumes TextBox only accepts digits. If it allows other characters (like -), int() could still raise, so wrap it in a try/except ValueError as well.

Prompt 2 — You

Now add a dynamic search range display. Track the minimum and maximum values still possible based on previous guesses a "too low" guess raises the minimum a "too high" guess lowers the maximum. Show it on screen as Range: 34 - 67 Reset the bounds in reset() too. The window is 620x380 and there is free space around y=270 so place it where it wont overlap the existing text

Claude

Four spots change: __init__, submit_guess, reset, and render.

__init__ (add after self.game_won = False):

```
self.min_possible = 1
self.max_possible = 100
self.font_range = pygame.font.SysFont(None, 28)
```

submit_guess

```
def submit_guess(self):
```

```

if self.game_won:
    return

text = self.input_box.text.strip()
if not text:
    self.feedback_msg = "Please enter a number first!"
    self.feedback_color = (255, 200, 60)
    return

guess = int(text)

self.attempts += 1
self.input_box.clear()

if guess < self.secret_number:
    self.feedback_msg = f"TOO LOW! (Guess was {guess})"
    self.feedback_color = (80, 160, 240)
    self.min_possible = max(self.min_possible, guess + 1)
elif guess > self.secret_number:
    self.feedback_msg = f"TOO HIGH! (Guess was {guess})"
    self.feedback_color = (240, 100, 80)
    self.max_possible = min(self.max_possible, guess - 1)
else:
    self.feedback_msg = f"CORRECT! Found in {self.attempts} attempts."
    self.feedback_color = (80, 220, 90)
    self.game_won = True

```

reset

```

def reset(self):
    self.secret_number = random.randint(1, 100)
    self.attempts = 0
    self.feedback_msg = "Enter a number between 1 and 100"
    self.feedback_color = (220, 220, 220)
    self.game_won = False
    self.min_possible = 1
    self.max_possible = 100
    self.input_box.clear()

```

render (insert right after the feedback blit, before the if self.game_won: block):

```

range_surf = self.font_range.render(
    f"Range: {self.min_possible} - {self.max_possible}", True, (180, 185, 195)
)
screen.blit(range_surf, (self.width // 2 - range_surf.get_width() // 2, 265))

```

The range line sits at y=265. The feedback text occupies roughly y=235-255 and the restart prompt starts at y=295, so it fits between them without overlapping either.

The max() / min() calls mean an out-of-range guess like 0 or 150 can never widen the bounds.

Prompt 3 — You

Add a guess history panel showing the last 5 guesses newest first each with a colour-coded arrow a red up arrow for "too high" and a blue down arrow for "too low"

Layout constraints: the window is 620x380 (WIDTH and HEIGHT in main.py)

Already used: title y=35, attempts y=95, input box and SUBMIT button y=150-198, feedback y=235, range y=265, restart message y=295.

Widen the window to 820x380 and draw the history as a panel down the right side (x from about 630 to 800) so nothing overlaps. Tell me exactly what to change in main.py too. Also clear the history in reset().

Claude

Changes go in __init__ , submit_guess , reset and render , plus one line in main.py .

main.py

```

WIDTH = 820    # was 620
HEIGHT = 380   # unchanged

```

I haven't seen main.py , so this assumes WIDTH and HEIGHT feed both pygame.display.set_mode((WIDTH, HEIGHT)) and GameEngine(WIDTH, HEIGHT) . If 620 is hardcoded anywhere else, change it there too.

`__init__`

The engine now receives 820 as its width, so the old `self.width // 2` centering would shift all the existing content right. Pin the centre to the original 620px play area instead:

```
self.width = width
self.height = height
self.play_width = 620
self.cx = self.play_width // 2    # centre of the left play area
# ... existing lines ...
self.history = []                  # newest first: (guess, "high" | "low")

self.input_box = TextBox(self.cx - 110, 150, 120, 48)
self.submit_btn = pygame.Rect(self.cx + 25, 150, 100, 48)
```

`submit_guess`

```
def submit_guess(self):
    if self.game_won:
        return

    text = self.input_box.text.strip()
    if not text:
        self.feedback_msg = "Please enter a number first!"
        self.feedback_color = (255, 200, 60)
        return

    guess = int(text)

    self.attempts += 1
    self.input_box.clear()

    if guess < self.secret_number:
        self.feedback_msg = f"TOO LOW! (Guess was {guess})"
        self.feedback_color = (80, 160, 240)
        self.min_possible = max(self.min_possible, guess + 1)
        self.history.insert(0, (guess, "low"))
    elif guess > self.secret_number:
        self.feedback_msg = f"TOO HIGH! (Guess was {guess})"
        self.feedback_color = (240, 100, 80)
        self.max_possible = min(self.max_possible, guess - 1)
        self.history.insert(0, (guess, "high"))
    else:
        self.feedback_msg = f"CORRECT! Found in {self.attempts} attempts."
        self.feedback_color = (80, 220, 90)
        self.game_won = True

    del self.history[5:]    # keep only the 5 most recent
```

`reset (add one line)`

```
self.history = []
```

`New helper method`

I draw the arrows as polygons because `SysFont(None)` often lacks ▲/▼ glyphs and would render empty boxes.

```
def _draw_arrow(self, screen, cx, cy, up, color):
    if up:
        points = [(cx, cy - 9), (cx - 8, cy + 7), (cx + 8, cy + 7)]
    else:
        points = [(cx, cy + 9), (cx - 8, cy - 7), (cx + 8, cy - 7)]
    pygame.draw.polygon(screen, color, points)
```

`render (full method, with self.cx replacing self.width // 2 and the panel at the end)`

```
def render(self, screen):
    screen.fill((30, 34, 42))

    title_surf = self.font_title.render("Number Guessing Arena", True, (245, 245, 245))
    screen.blit(title_surf, (self.cx - title_surf.get_width() // 2, 35))

    attempts_surf = self.font_medium.render(f"Attempts: {self.attempts}", True, (180, 185, 195))
    screen.blit(attempts_surf, (self.cx - attempts_surf.get_width() // 2, 95))
    self.input_box.render(screen)
```

```

pygame.draw.rect(screen, (50, 150, 80), self.submit_btn, border_radius=6)
pygame.draw.rect(screen, (220, 220, 220), self.submit_btn, width=2, border_radius=6)
btn_text = self.font_btn.render("SUBMIT", True, (255, 255, 255))
screen.blit(
    btn_text,
    (self.submit_btn.centerx - btn_text.get_width() // 2,
     self.submit_btn.centery - btn_text.get_height() // 2),
)

feedback_surf = self.font_medium.render(self.feedback_msg, True, self.feedback_color)
screen.blit(feedback_surf, (self.cx - feedback_surf.get_width() // 2, 235))

range_surf = self.font_range.render(
    f"Range: {self.min_possible} - {self.max_possible}", True, (180, 185, 195)
)
screen.blit(range_surf, (self.cx - range_surf.get_width() // 2, 265))

if self.game_won:
    restart_surf = self.font_medium.render("Press [R] to Start a New Game", True, (255, 220,
80))
    screen.blit(restart_surf, (self.cx - restart_surf.get_width() // 2, 295))

# --- History panel (x 630-800) ---
panel = pygame.Rect(630, 35, 170, 300)
pygame.draw.rect(screen, (40, 45, 56), panel, border_radius=8)
pygame.draw.rect(screen, (90, 95, 105), panel, width=2, border_radius=8)

header = self.font_medium.render("History", True, (245, 245, 245))
screen.blit(header, (panel.centerx - header.get_width() // 2, panel.y + 14))

if not self.history:
    empty = self.font_btn.render("No guesses yet", True, (130, 135, 145))
    screen.blit(empty, (panel.centerx - empty.get_width() // 2, panel.y + 70))

for i, (guess, direction) in enumerate(self.history):
    row_cy = panel.y + 75 + i * 46
    color = (240, 100, 80) if direction == "high" else (80, 160, 240)
    self._draw_arrow(screen, panel.x + 35, row_cy, direction == "high", color)
    num_surf = self.font_medium.render(str(guess), True, color)
    screen.blit(num_surf, (panel.x + 65, row_cy - num_surf.get_height() // 2))

```

The panel spans x=630-800 and y=35-335, and the game content stays centred at x=310, so the widest text (the title, about 330px) ends near x=475 and nothing overlaps. History rows start at y=110 and the fifth ends around y=290.

Only "too high" and "too low" guesses are recorded, as you specified, so the winning guess isn't listed. If you want it shown too, add a green check row in the else branch.

Prompt 4 — You

Add a maximum of 10 attempts. When the player runs out, show a Game Over screen that reveals the secret number, stop accepting further guesses, and allow R to start a new game. Update the attempts display to read "Attempts: 3 / 10", and reset the new state in `reset()`.

Note: `handle_event` currently only allows R when `self.game_won` is True, so that condition needs to change as well, or the player will be stuck on Game Over.

Claude

Five spots change: `__init__`, `submit_guess`, `reset`, `handle_event`, and `render`.

`__init__` (add after `self.game_won = False`)

```

self.max_attempts = 10
self.game_over = False

```

`submit_guess`

```

def submit_guess(self):
    if self.game_won or self.game_over:
        return

    text = self.input_box.text.strip()
    if not text:
        self.feedback_msg = "Please enter a number first!"

```

```

        self.feedback_color = (255, 200, 60)
        return

    guess = int(text)

    self.attempts += 1
    self.input_box.clear()

    if guess < self.secret_number:
        self.feedback_msg = f"TOO LOW! (Guess was {guess})"
        self.feedback_color = (80, 160, 240)
        self.min_possible = max(self.min_possible, guess + 1)
        self.history.insert(0, (guess, "low"))
    elif guess > self.secret_number:
        self.feedback_msg = f"TOO HIGH! (Guess was {guess})"
        self.feedback_color = (240, 100, 80)
        self.max_possible = min(self.max_possible, guess - 1)
        self.history.insert(0, (guess, "high"))
    else:
        self.feedback_msg = f"CORRECT! Found in {self.attempts} attempts."
        self.feedback_color = (80, 220, 90)
        self.game_won = True

    del self.history[5:] # keep only the 5 most recent

    if not self.game_won and self.attempts >= self.max_attempts:
        self.game_over = True
        self.feedback_msg = f"GAME OVER! The number was {self.secret_number}"
        self.feedback_color = (240, 100, 80)

```

The out-of-attempts check runs after the guess is evaluated, so a correct 10th guess still counts as a win.

reset (add one line)

```

    self.game_over = False

```

handle_event (only the R condition changes)

```

        elif event.key == pygame.K_r and (self.game_won or self.game_over):
            self.reset()

```

render

Replace the attempts line:

```

    attempts_surf = self.font_medium.render(
        f"Attempts: {self.attempts} / {self.max_attempts}", True, (180, 185, 195)
    )

```

Then add this at the very end of render , after the history panel code:

```

# --- Game Over overlay (covers only the 620px play area) ---
if self.game_over:
    overlay = pygame.Surface((self.play_width, self.height), pygame.SRCALPHA)
    overlay.fill((20, 22, 28, 225))
    screen.blit(overlay, (0, 0))

    over_surf = self.font_title.render("GAME OVER", True, (240, 100, 80))
    screen.blit(over_surf, (self.cx - over_surf.get_width() // 2, 110))

    label_surf = self.font_medium.render("The secret number was", True, (180, 185, 195))
    screen.blit(label_surf, (self.cx - label_surf.get_width() // 2, 170))

    num_surf = self.font_title.render(str(self.secret_number), True, (255, 220, 80))
    screen.blit(num_surf, (self.cx - num_surf.get_width() // 2, 200))

    restart_surf = self.font_medium.render("Press [R] to Start a New Game", True, (245, 245,
        245))
    screen.blit(restart_surf, (self.cx - restart_surf.get_width() // 2, 270))

```

The overlay only dims the left play area, so the history panel stays visible and you can still see the final guesses that led to the loss. The overlay also hides the input box and SUBMIT button, and submit_guess returns early in the game-over state, so clicking where the button was does nothing.